EDA(Explortary Data Analysis)

Univariant Analysis

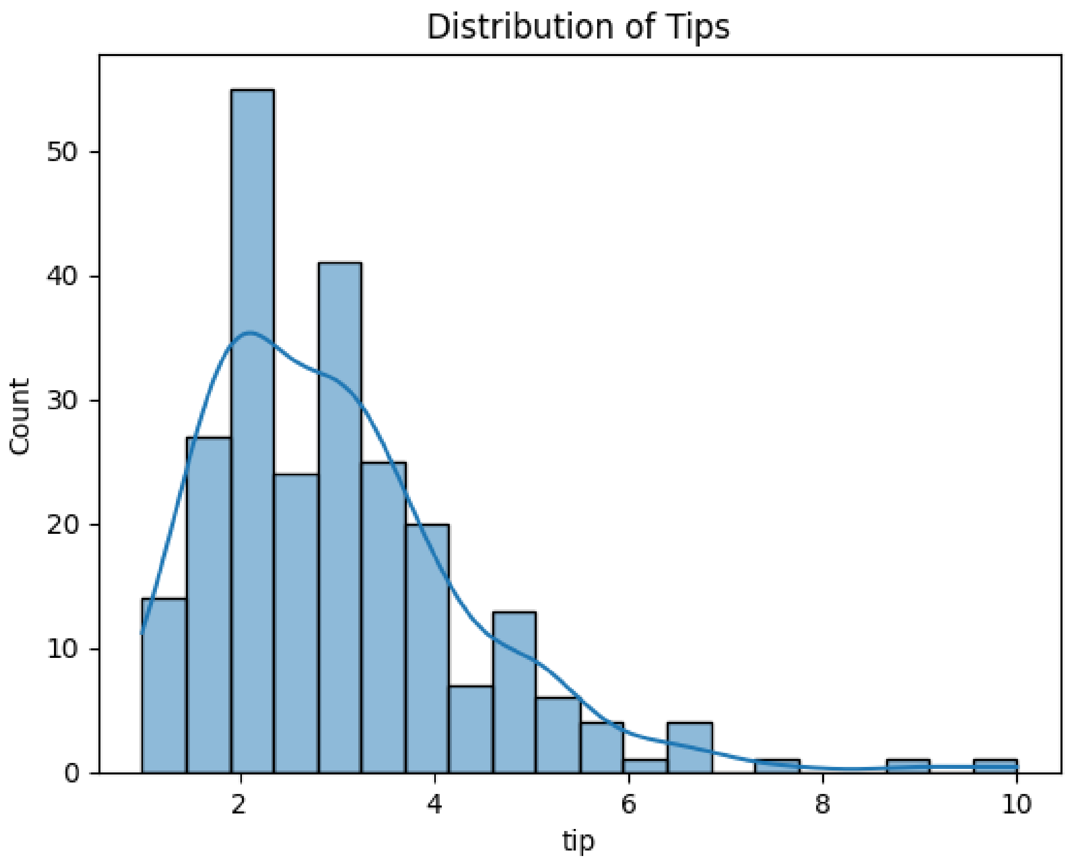
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset("tips")

print(df.head())
print(df.info())
print(df.columns)
```

```
   total_bill   tip     sex smoker  day    time  size
0       16.99  1.01  Female     No  Sun  Dinner     2
1       10.34  1.66    Male     No  Sun  Dinner     3
2       21.01  3.50    Male     No  Sun  Dinner     3
3       23.68  3.31    Male     No  Sun  Dinner     2
4       24.59  3.61  Female     No  Sun  Dinner     4
<class 'pandas.DataFrame'>
RangeIndex: 244 entries, 0 to 243
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   total_bill  244 non-null    float64
 1   tip         244 non-null    float64
 2   sex         244 non-null    category
 3   smoker      244 non-null    category
 4   day         244 non-null    category
 5   time        244 non-null    category
 6   size        244 non-null    int64
dtypes: category(4), float64(2), int64(1)
memory usage: 7.4 KB
None
Index(['total_bill', 'tip', 'sex', 'smoker', 'day', 'time', 'size'],
dtype='str')
```

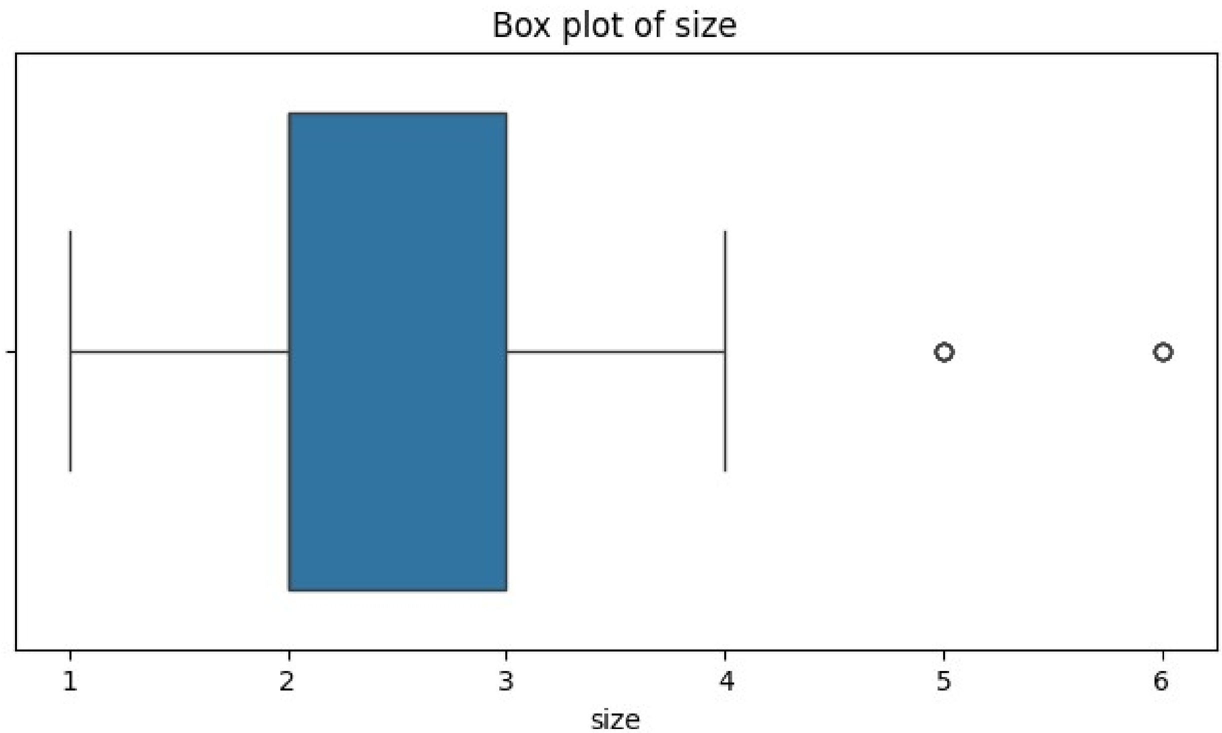
```
# Descriptive Statistics
print(df['total_bill'].describe())
```

```
count    244.000000
mean      19.785943
std        8.902412
min        3.070000
25%       13.347500
```

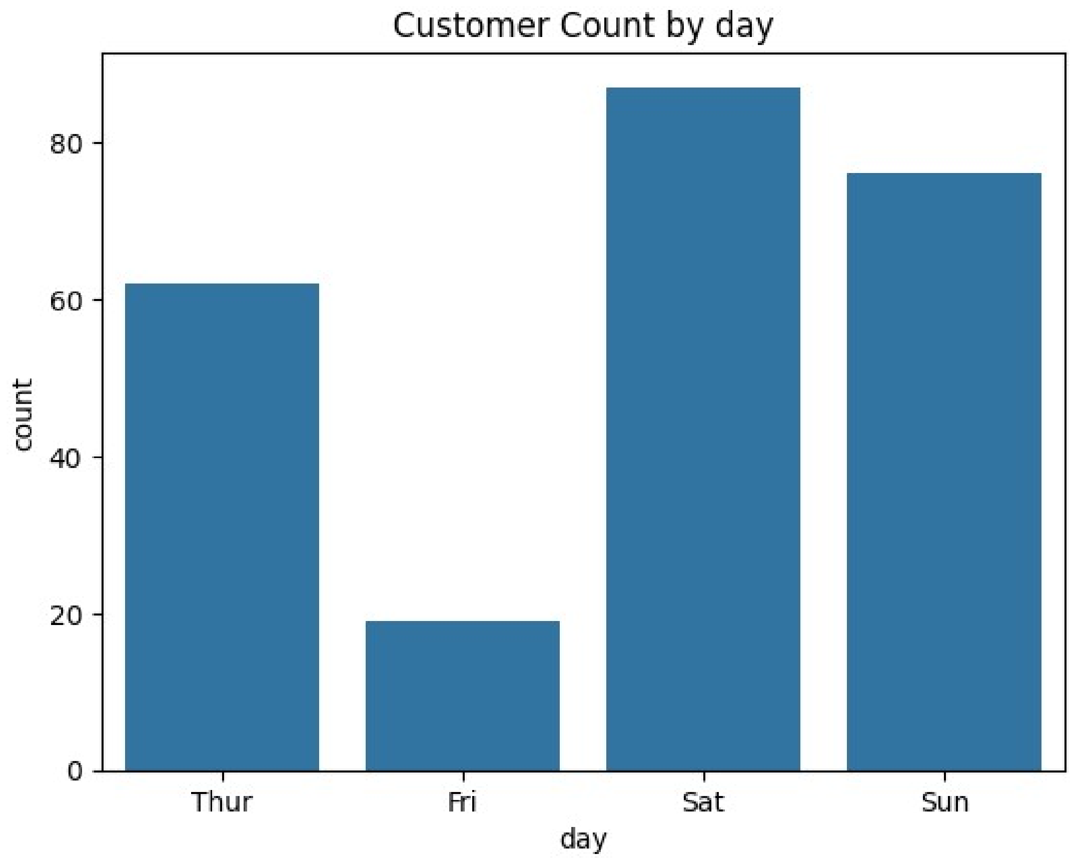
```
50%         17.795000
75%         24.127500
max         50.810000
Name: total_bill, dtype: float64

# 2. Histogram
sns.histplot(df['tip'],bins=20, kde=True)
plt.title("Distribution of Tips")
plt.show()
```



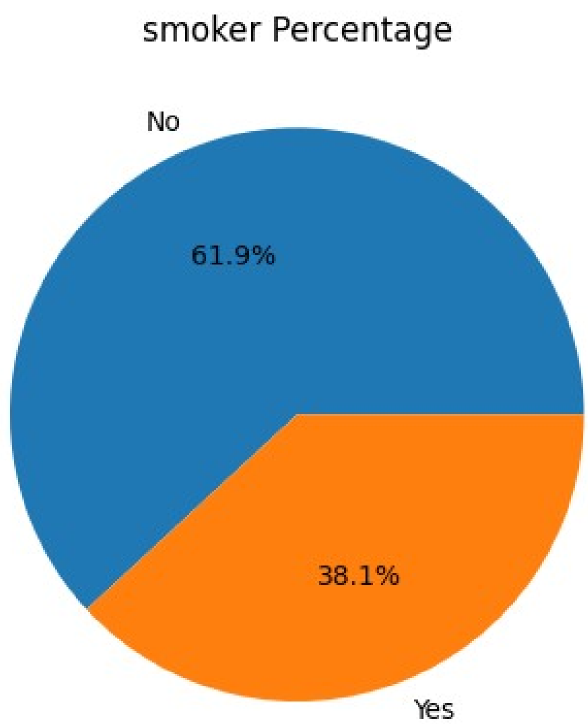

```
# Box plot
plt.figure(figsize=(8,4))
sns.boxplot(x=df['size'])
plt.title("Box plot of size")
plt.show()
```




```
# Categorical variable:Day
print(df['day'].value_counts())

day
Sat     87
Sun     76
Thur    62
Fri     19
Name: count, dtype: int64

# count plot
sns.countplot(x='day',data=df)
plt.title("Customer Count by day")
plt.show()
```




```
# pie chart
df['smoker'].value_counts().plot(
    kind='pie',
    autopct='%1.1f%%'
)

plt.title("smoker Percentage")
plt.ylabel("")
plt.show()
```

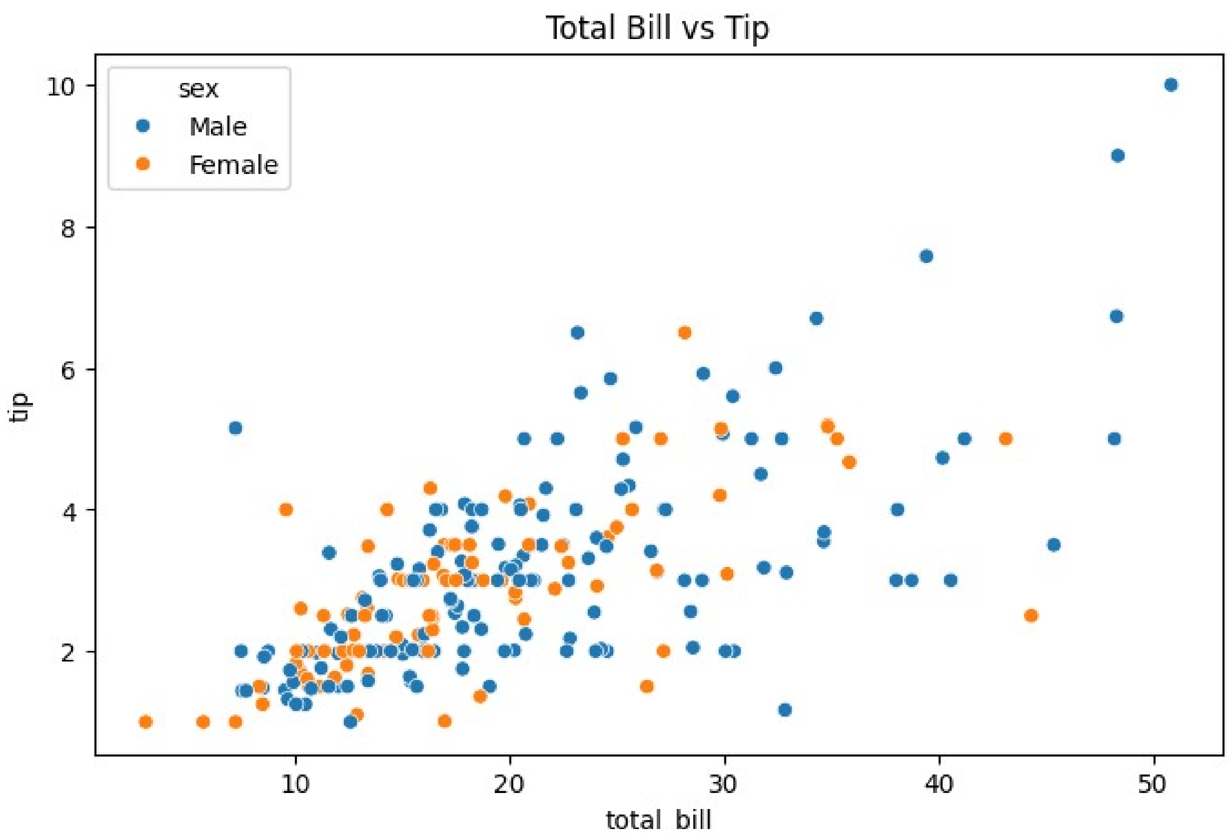



```
##Outlier Detection Using IQR
Q1 = df['total_bill'].quantile(0.25)
Q3 = df['total_bill'].quantile(0.75)


IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q1 + 1.5 * IQR
outliers_iqr =df[
    (df['total_bill'] <lower) |
    (df['total_bill']>upper)]

print("outliers:",len(outliers_iqr))

outliers: 35
```

Bivariant Analysis

```
#numeric vs numeric
plt.figure(figsize=(8,5))
sns.scatterplot(
    x ='total_bill',
    y ='tip',
```

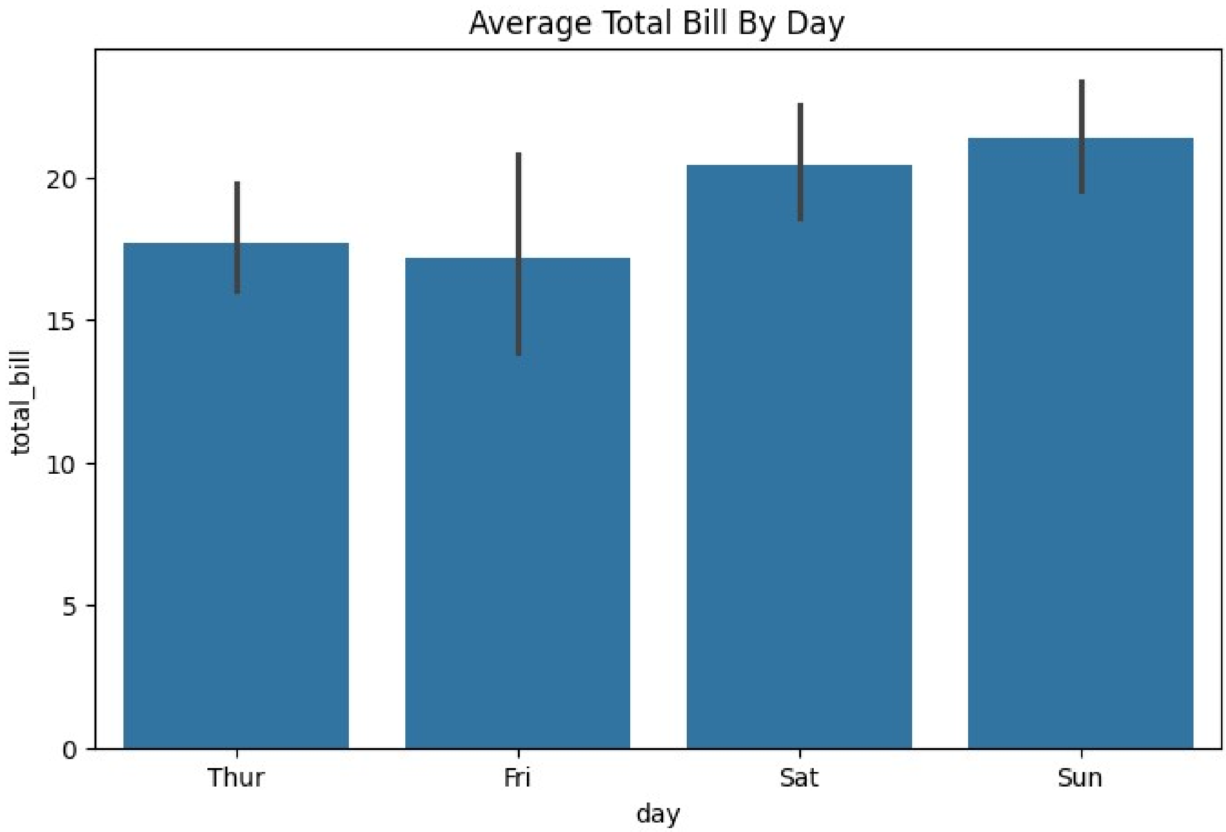
```
    hue ='sex',
    data =df
)
plt.title("Total Bill vs Tip")
plt.show()
```




```
# correlation
#pearson correlation
corr = df['total_bill'].corr(df['tip'])
print(corr)
```

```
0.6757341092113645
```

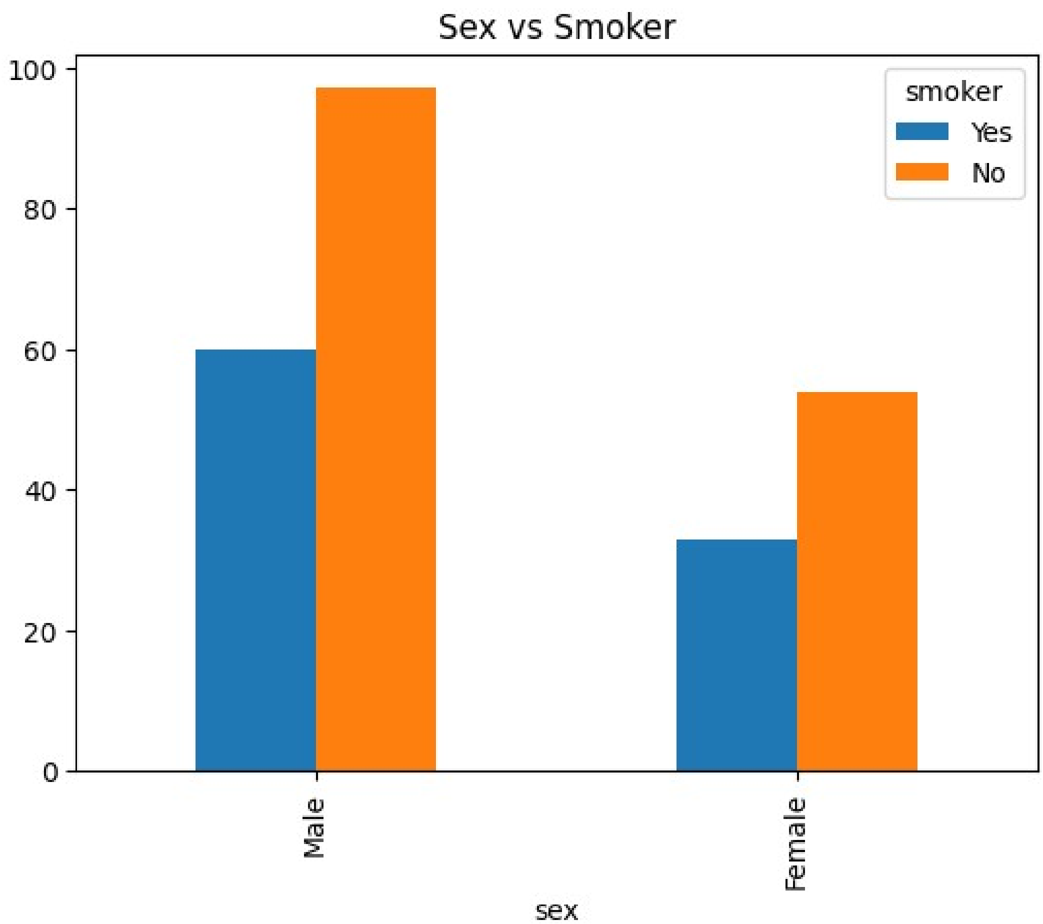
```
# Categorical vs numerical
#bar chart
plt.figure(figsize=(8,5))
sns.barplot(
    x ='day',
    y ='total_bill',
    data =df
)
plt.title("Average Total Bill By Day")
plt.show()
```




```
#Grouped Statistics
#mean Bill by Day
print(
    df.groupby("day",observed=False)['total_bill'].mean()
)
# Multiple statistics
print(
    df.groupby('day',observed=False)['total_bill']
    .agg(['mean','median','min','max'])
)
```

```
day
Thur    17.682742
Fri     17.151579
Sat     20.441379
Sun     21.410000
Name: total_bill, dtype: float64
           mean  median   min    max
day
Thur  17.682742   16.20  7.51  43.11
Fri   17.151579   15.38  5.75  40.17
Sat   20.441379   18.24  3.07  50.81
Sun   21.410000   19.63  7.25  48.17
```

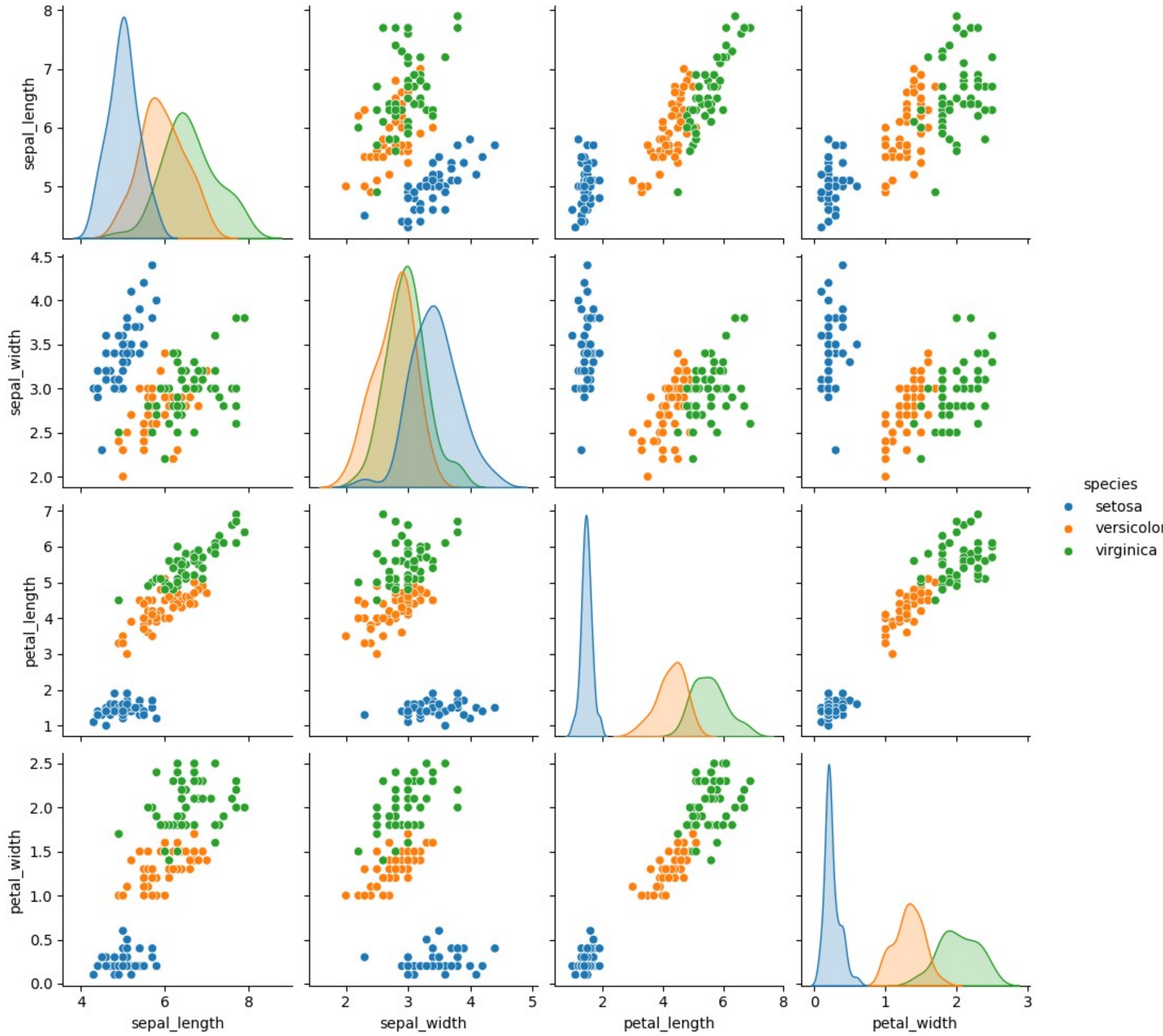
```
#categorical vs categorical
# Bar Chart
pd.crosstab(
    df['sex'],
    df['smoker']
).plot(kind="bar")
plt.title("Sex vs Smoker")
plt.show()
```



Multivariant analysis

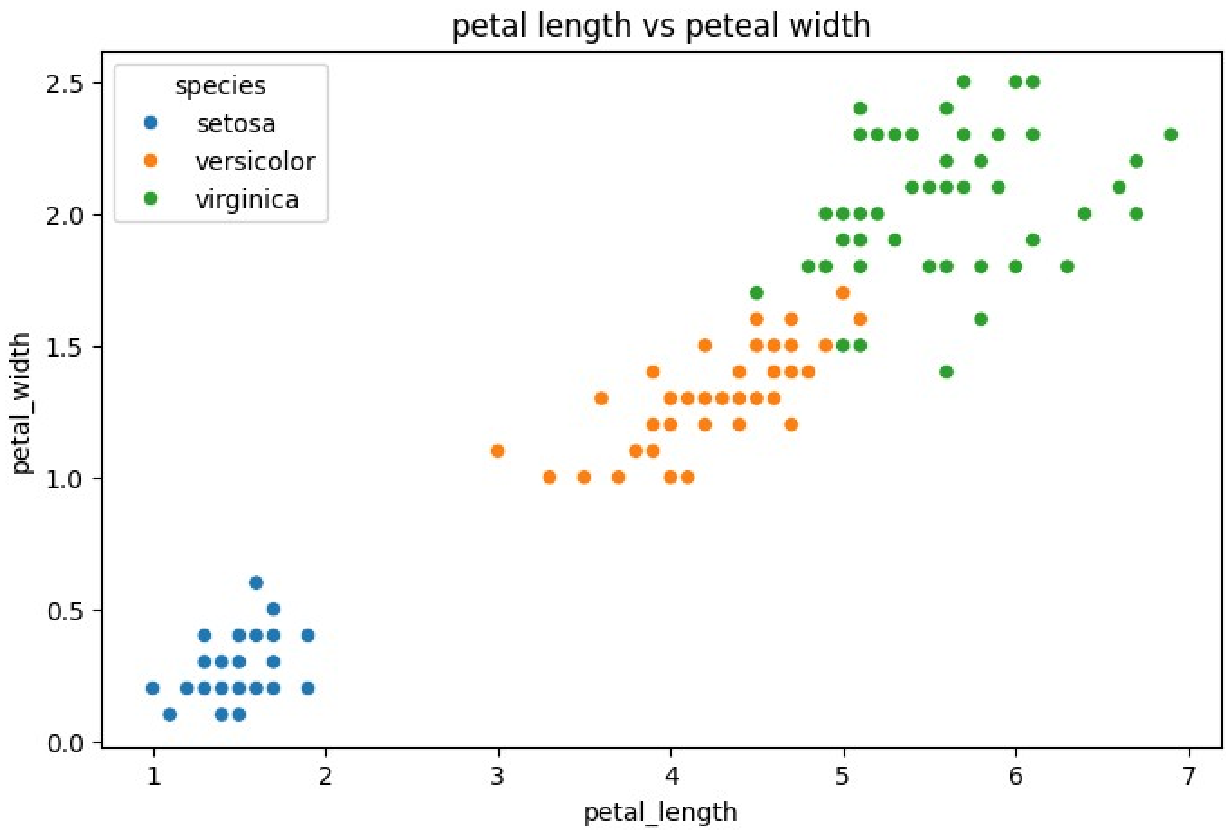
```
# pairplot
import seaborn as sns
import matplotlib.pyplot as plt
df = sns.load_dataset("iris")
sns.pairplot(
    df,
    hue='species'
)
```

```
plt.show()
print(df.columns)
```

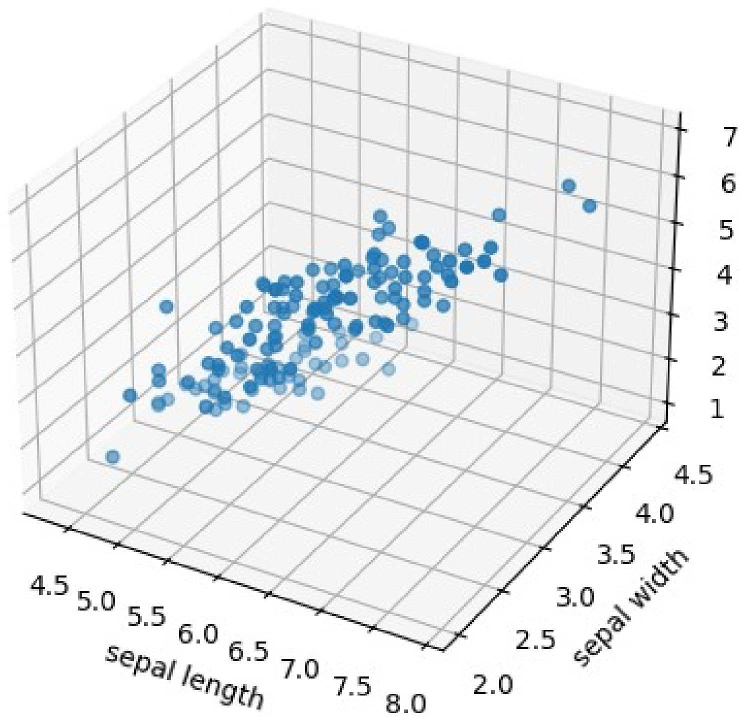



```
Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width',
       'species'],
      dtype='str')

#colour -coded scatter plot
plt.figure(figsize=(8,5))
sns.scatterplot(
    x='petal_length',
    y='petal_width',
    hue='species',
    data=df
)
```

```
plt.title("petal length vs peteal width")
plt.show()
```

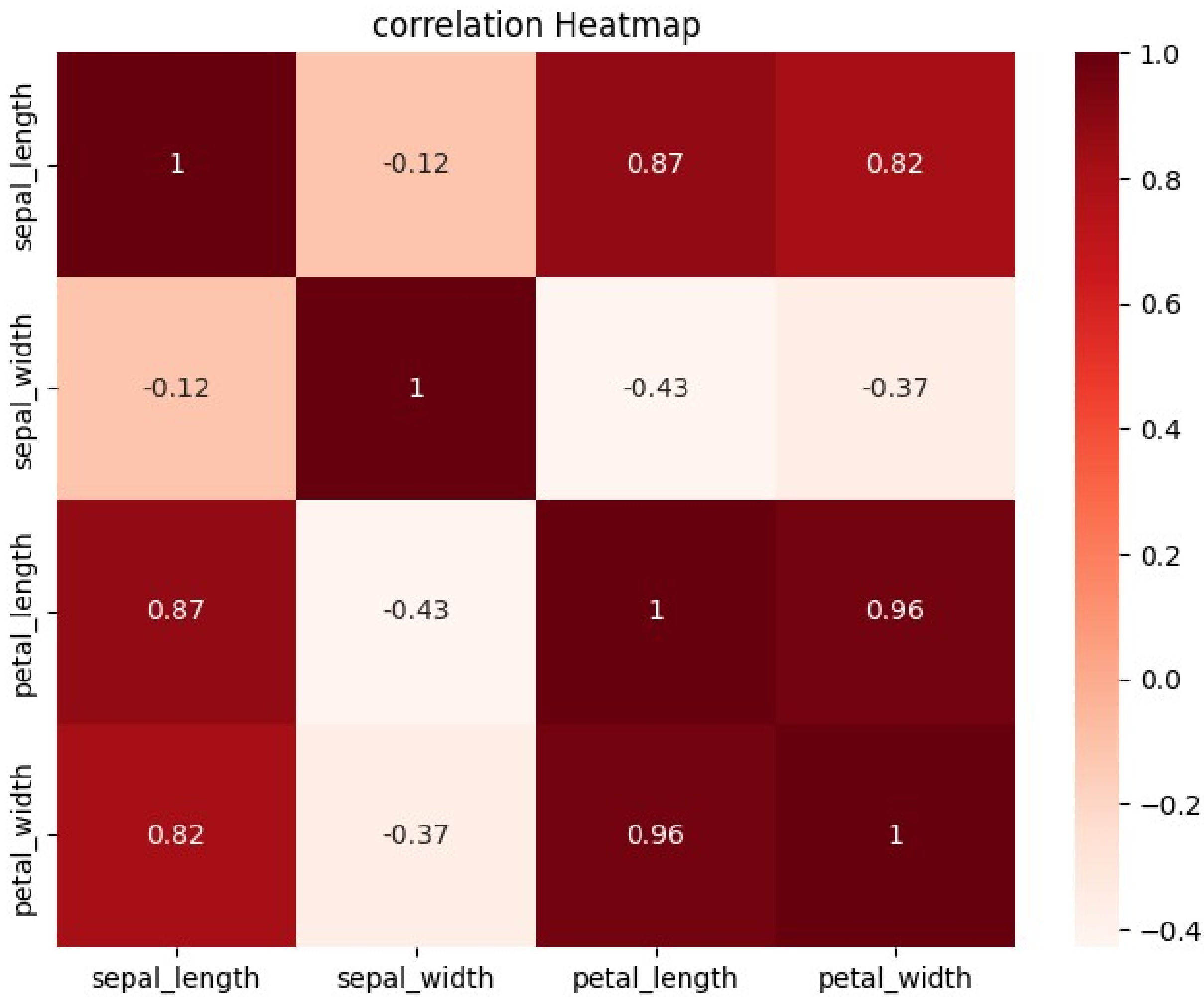



```
#3D Scatter plot
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt
fig=plt.figure(figsize=(5,5))
ax=fig.add_subplot(111,projection='3d')
ax.scatter(
    df['sepal_length'],
    df['sepal_width'],
    df['petal_length']
)
ax.set_xlabel('sepal length')
ax.set_ylabel('sepal width')
ax.set_zlabel('petal_length')
plt.show()
```



Correlation Analysis multi collinearity

```
corr_matrix=df.corr(numeric_only=True)
print(corr_matrix)

              sepal_length  sepal_width  petal_length  petal_width
sepal_length      1.000000    -0.117570      0.871754     0.817941
sepal_width      -0.117570     1.000000     -0.428440    -0.366126
petal_length      0.871754    -0.428440      1.000000     0.962865
petal_width       0.817941    -0.366126      0.962865     1.000000

plt.figure(figsize=(8,6))
sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    cmap='Reds'
)
plt.title("correlation Heatmap")
plt.show()
```



VIF(Variance inflattion Factor)

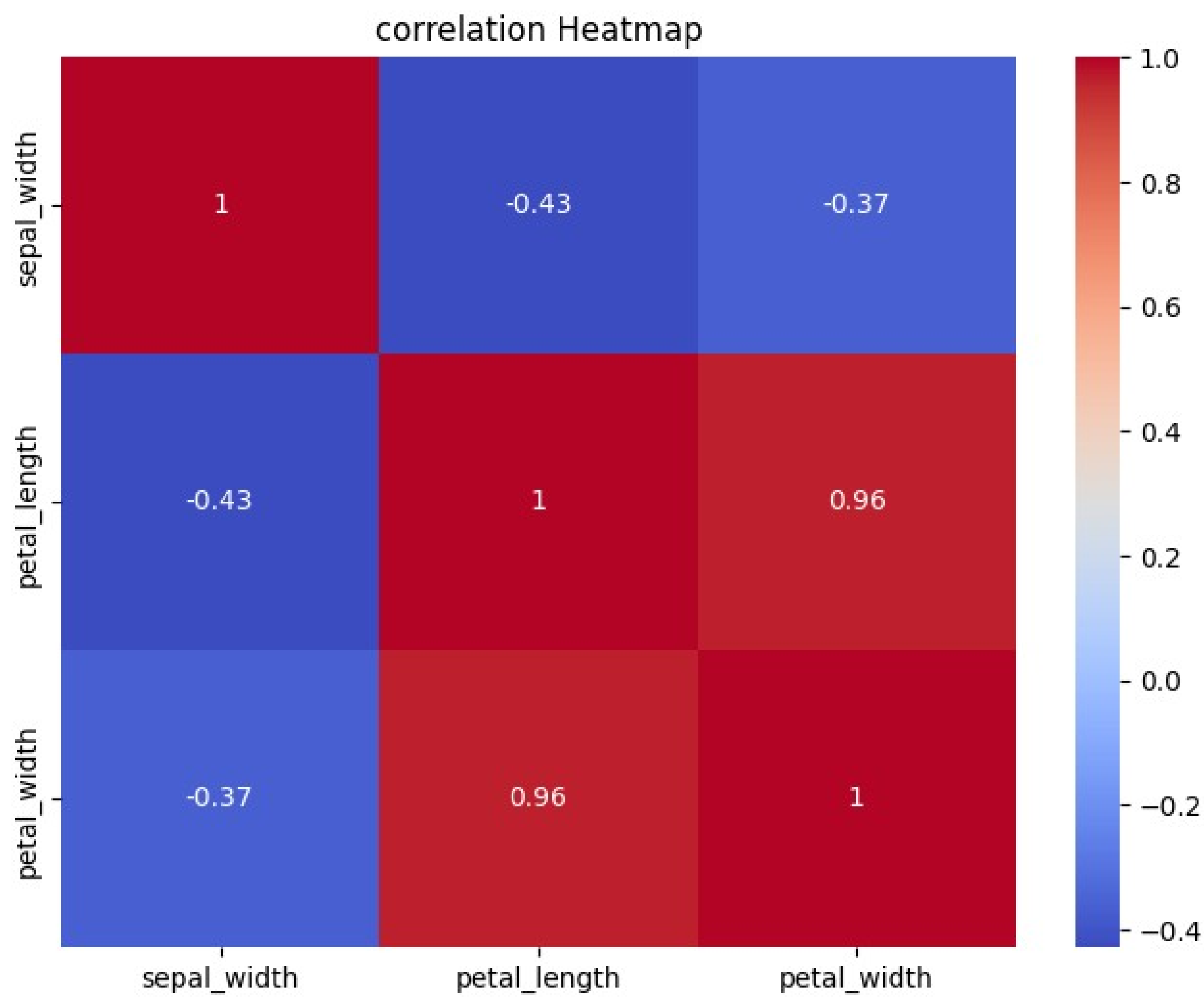
```
from statsmodels.stats.outliers_influence import
variance_inflation_factor
x=df.drop("species",axis=1)
vif=pd.DataFrame()
vif["Feature"]= x.columns
vif["VIF"] = [
    variance_inflation_factor(x.values,i)
    for i in range(x.shape[1])
]
print(vif)
```

```
        Feature         VIF
0  sepal_length  262.969348
1   sepal_width   96.353292
2  petal_length  172.960962
3   petal_width   55.502060
```

```python
#remove a high VIF Feature
#support we remove petal_length.
X_new = x.drop("sepal_length",axis=1)
vif_after = pd.DataFrame()
vif_after["Feature"] = X_new.columns
vif_after["VIF"] = [
    variance_inflation_factor(X_new.values,i)
    for i in range(X_new.shape[1])
]
print(vif_after)
```

```
        Feature        VIF
0   sepal_width   5.856965
1  petal_length  62.071308
2   petal_width  43.292574
```

```python
#heatmap after removing feature of sepal_length
plt.figure(figsize=(8,6))
sns.heatmap(
    X_new.corr(),
    annot=True,
    cmap='coolwarm'
)
plt.title("correlation Heatmap")
plt.show()
```


correlation Heatmap
sepal_width
petal_length
petal_width
1
-0.43
-0.37
-0.43
1
0.96
-0.37
0.96
1
sepal_width
petal_length
petal_width
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4